

tum machine learning algorithms include the quantum support vector machine (QSVM), quantum principal component analysis (QPCA), and quantum k-means. One example of a quantum machine learning algorithm is the quantum approximate optimisation algorithm (QAOA), which is used to solve optimisation problems. QAOA is a hybrid algorithm that combines classical optimisation with quantum operations to find the optimal solution to a problem.

Another example of a quantum machine learning technique is a quantum-inspired classical algorithm. These algorithms are designed to mimic the behaviour of quantum systems using classical computers, with the potential for improved performance in certain tasks.

Platforms and libraries for quantum machine learning

As already stated, QML is an interdisciplinary research area at the intersection of quantum computing and machine learning. In recent years, several libraries and platforms have emerged to facilitate the development of QML algorithms and applications. Here are some popular ones.

TensorFlow Quantum (TFQ)

<https://www.tensorflow.org/quantum>

TFQ is a library developed by Google that enables the creation of quantum machine learning models in TensorFlow. It provides a high-level interface for constructing quantum circuits and integrating them into classical machine learning models.

PennyLane

<https://pennylane.ai>

PennyLane is an open source software library for building and training quantum machine learning models. It provides a unified interface to different quantum hardware and simulators, allowing researchers to develop and test their algorithms on a range of platforms.

Qiskit Machine Learning

<https://qiskit.org/ecosystem/machine-learning>

Qiskit is an open source framework for programming quantum computers, and Qiskit Machine Learning is an extension that adds quantum machine learning algorithms to the toolkit. It provides a range of machine learning tools, including classical machine learning models that can be trained on quantum data.

Pyquil

<https://pyquil-docs.rigetti.com/en/stable>

Pyquil is a library for quantum programming in Python, developed by Rigetti Computing. It provides a simple interface for constructing and simulating quantum circuits and allows for the creation of hybrid quantum-classical models for machine learning. Forest is a suite

of software tools for developing and running quantum applications, also developed by Rigetti Computing. It includes Pyquil and other tools for quantum programming, as well as a cloud-based platform for running quantum simulations and experiments.

IBM Q Experience

<https://quantum-computing.ibm.com>

IBM Q Experience is a cloud-based platform for programming and running quantum circuits on IBM's quantum computers. It includes a range of tools for building and testing quantum algorithms, including quantum machine learning algorithms.

These are just some of the platforms and libraries available for quantum machine learning. As the field continues to grow, we can expect to see more tools and platforms emerge to support this exciting field of research.

Implementation scenarios

The following code demonstrates how to use a quantum circuit to classify two data points as either class 0 or class 1, based on a training data set.

```
from qiskit import QuantumCircuit, execute, Aer
from qiskit.aqua.components.feature_maps import
RawFeatureVector

# Define training data and labels
training_data = [[1, 0], [0, 1]]
training_labels = [0, 1]

# Define feature map circuit
feature_map = RawFeatureVector(feature_dimension=2, data_map_
func=lambda x: x)

# Define quantum circuit
qc = QuantumCircuit(2, 1)
qc.append(feature_map, [0, 1])
qc.h(0)
qc.cx(0, 1)

# Measure qubit 0 to obtain classification result
qc.measure(0, 0)

# Execute circuit on local simulator
backend = Aer.get_backend('qasm_simulator')
job = execute(qc, backend, shots=1024)
result = job.result()

# Print classification result
counts = result.get_counts()
print(counts)
```

In this code, we first define a training data set with